

BÁO CÁO DỰ ÁN CUỐI KỲ OJT_FA25

Môn học: On-the-Job Training (OJT) – FA25

Đề tài: Xây dựng và triển khai hệ thống phân loại hình ảnh đa lớp dựa trên bộ dữ liệu CIFAR-10 sử dụng mạng nơ-ron tích chập sâu

Nguyễn Thị Minh Thu

Mã số sinh viên: SE185043

Lớp: OJT_FA25

Link Github: [MinhThuNT/CIFAR-10-Image-Classification](https://github.com/MinhThuNT/CIFAR-10-Image-Classification)

Thời gian thực hiện: Tháng 09 – 12/2025

TÓM TẮT

Báo cáo trình bày toàn bộ quy trình phát triển một hệ thống phân loại hình ảnh hoàn chỉnh dựa trên bộ dữ liệu chuẩn CIFAR-10 (10 lớp đối tượng). Mô hình mạng nơ-ron tích chập (CNN) được thiết kế theo phong cách VGG cải tiến, kết hợp các kỹ thuật hiện đại như Batch Normalization, L2 regularization và Data Augmentation mạnh mẽ, đạt độ chính xác 90,68% trên tập kiểm tra – thuộc nhóm kết quả cao nhất với kiến trúc tự xây dựng.

Hệ thống được triển khai end-to-end bao gồm: (1) cơ chế huấn luyện ổn định với checkpoint tự động và khả năng tiếp tục (resume) khi bị gián đoạn, (2) hàm dự đoán ảnh thực tế ngoài tập dữ liệu, và (3) ứng dụng web thời gian thực sử dụng Flask, HTML/CSS/JavaScript với giao diện người dùng hiện đại, hỗ trợ kéo-thả ảnh và hiển thị top-3 dự đoán.

Kết quả cho thấy mô hình không chỉ đạt hiệu năng cao trong môi trường học thuật mà còn có khả năng hoạt động tốt trên ảnh thực tế, đáp ứng đầy đủ yêu cầu triển khai sản phẩm thực tế của chương trình OJT.

Từ khóa: CIFAR-10, Convolutional Neural Network, Data Augmentation, Flask Web Application, Model Checkpointing.

1. GIỚI THIỆU

Phân loại hình ảnh là một trong những bài toán nền tảng của thị giác máy tính. Bộ dữ liệu CIFAR-10 (Krizhevsky, 2009) với 60.000 ảnh màu 32×32 thuộc 10 lớp được sử dụng rộng rãi để đánh giá hiệu năng của các kiến trúc học sâu.

Dự án đặt mục tiêu không chỉ đạt độ chính xác cao mà còn xây dựng một quy trình phát triển hoàn chỉnh, bền vững và có khả năng triển khai thực tế – yếu tố quan trọng trong môi trường doanh nghiệp.

2. MỤC TIÊU NGHIÊN CỨU

1. Thiết kế và huấn luyện một mô hình CNN đạt độ chính xác $\geq 90\%$ trên CIFAR-10 mà không sử dụng kiến trúc tiền huấn luyện.

- 2. Xây dựng cơ chế huấn luyện ổn định với khả năng tự động lưu và tiếp tục từ checkpoint.
- 3. Phát triển hàm dự đoán mạnh mẽ cho ảnh thực tế (out-of-distribution).
- 4. Triển khai ứng dụng web thời gian thực với giao diện thân thiện và hiệu năng cao.

3. PHƯƠNG PHÁP THỰC HIỆN

3.1. Dữ liệu và tiền xử lý

Trong nghiên cứu, bộ dữ liệu CIFAR-10 được sử dụng gồm 50.000 ảnh cho huấn luyện và 10.000 ảnh cho kiểm tra. Từ tập huấn luyện, 10% dữ liệu được tách ra để tạo thành tập validation, dẫn đến phân chia cuối cùng là 45.000 ảnh cho huấn luyện, 5.000 ảnh cho validation và 10.000 ảnh cho kiểm tra. Dữ liệu đầu vào được chuẩn hóa bằng Z-score, sử dụng giá trị trung bình và độ lệch chuẩn tính trên tập huấn luyện. Các nhãn được mã hóa theo dạng one-hot encoding nhằm phục vụ cho quá trình huấn luyện mô hình. Ngoài ra, áp dụng kỹ thuật tăng cường dữ liệu (Data Augmentation) thông qua ImageDataGenerator, bao gồm xoay ngẫu nhiên trong khoảng $\pm 15^\circ$, dịch chuyển ngang và dọc tối đa 12%, lật ngang, phóng to (zoom) 10%, cũng như điều chỉnh độ sáng và kênh màu. Những bước xử lý này giúp cải thiện khả năng tổng quát hóa của mô hình và giảm hiện tượng overfitting.

3.2. Kiến trúc mô hình

Mô hình được thiết kế theo phong cách VGG cải tiến, tối ưu cho bài toán phân loại ảnh kích thước nhỏ (32×32) như CIFAR-10. Kiến trúc gồm 8 lớp tích chập, xen kẽ với Batch Normalization để ổn định quá trình huấn luyện và Dropout nhằm giảm overfitting. Sau cùng là tầng Dense 10-class cho bài toán phân loại đa lớp.

Bảng dưới mô tả đầy đủ kiến trúc mô hình, kích thước đầu ra và số lượng tham số tại từng lớp:

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
batch_normalization (BatchNormalization)	(None, 32, 32, 32)	128
conv2d_1 (Conv2D)	(None, 32, 32, 32)	9,248

batch_normalization_1	(None, 32, 32, 32)	128
(BatchNormalization)		
max_pooling2d	(None, 16, 16, 32)	0
(MaxPooling2D)		
dropout (Dropout)	(None, 16, 16, 32)	0
conv2d_2 (Conv2D)	(None, 16, 16, 64)	18,496
batch_normalization_2	(None, 16, 16, 64)	256
(BatchNormalization)		
conv2d_3 (Conv2D)	(None, 16, 16, 64)	36,928
batch_normalization_3	(None, 16, 16, 64)	256
(BatchNormalization)		
max_pooling2d_1	(None, 8, 8, 64)	0
(MaxPooling2D)		
dropout_1 (Dropout)	(None, 8, 8, 64)	0
conv2d_4 (Conv2D)	(None, 8, 8, 128)	73,856
batch_normalization_4	(None, 8, 8, 128)	512
(BatchNormalization)		

conv2d_5 (Conv2D)	(None, 8, 8, 128)	147,584
batch_normalization_5 (BatchNormalization)	(None, 8, 8, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0
dropout_2 (Dropout)	(None, 4, 4, 128)	0
conv2d_6 (Conv2D)	(None, 4, 4, 256)	295,168
batch_normalization_6 (BatchNormalization)	(None, 4, 4, 256)	1,024
conv2d_7 (Conv2D)	(None, 4, 4, 256)	590,080
batch_normalization_7 (BatchNormalization)	(None, 4, 4, 256)	1,024
max_pooling2d_3 (MaxPooling2D)	(None, 2, 2, 256)	0
dropout_3 (Dropout)	(None, 2, 2, 256)	0
flatten (Flatten)	(None, 1024)	0

dense (Dense)	(None, 10)	10,250

Total params: 1,186,346 (4.53 MB)

Trainable params: 1,184,426 (4.52 MB)

Non-trainable params: 1,920 (7.50 KB)

a. Khối Convolution 1 – 32 filters

Bao gồm 2 lớp Conv2D với số lượng filter = 32, kernel kích thước 3×3 , padding = “same”.

Sau mỗi lớp Conv2D có Batch Normalization để ổn định phân phối đầu vào, giảm hiện tượng *internal covariate shift*.

Tiếp theo là MaxPooling 2×2 , giúp giảm kích thước không gian từ $32 \times 32 \rightarrow 16 \times 16$.

Cuối khối có Dropout (tỉ lệ 0.2) nhằm hạn chế overfitting bằng cách ngẫu nhiên bỏ bớt neuron trong quá trình huấn luyện.

b. Khối Convolution 2 – 64 filters

Cấu trúc tương tự khối 1 nhưng số lượng filter tăng lên 64, cho phép mô hình học được nhiều đặc trưng phức tạp hơn.

Sau 2 lớp Conv2D + BatchNorm là MaxPooling 2×2 , giảm kích thước từ $16 \times 16 \rightarrow 8 \times 8$.

Dropout được tăng lên 0.3 để tăng khả năng chống overfitting khi số lượng tham số nhiều hơn.

c. Khối Convolution 3 – 128 filters

Số filter tiếp tục tăng lên 128, giúp mô hình nhận diện được các đặc trưng chi tiết hơn như texture và pattern phức tạp.

Sau 2 lớp Conv2D + BatchNorm là MaxPooling 2×2 , giảm kích thước từ $8 \times 8 \rightarrow 4 \times 4$.

Dropout tăng lên 0.4, phù hợp với độ sâu và số lượng tham số lớn hơn.

d. Khối Convolution 4 – 256 filters

Đây là khối tích chập sâu nhất, gồm 2 lớp Conv2D với 256 filters.

Mỗi lớp đều có Batch Normalization để giữ gradient ổn định trong quá trình huấn luyện mạng sâu.

MaxPooling 2×2 cuối cùng giảm kích thước từ $4 \times 4 \rightarrow 2 \times 2$.

Dropout được đặt ở mức 0.5, giúp giảm thiểu hiện tượng co-adaptation giữa các neuron.

e. Fully Connected Layer

Flatten: chuyển tensor đầu ra từ khối convolution cuối cùng (kích thước $2 \times 2 \times 256$) thành vector phẳng 1.024 chiều.

Dense Layer: một lớp fully connected với 10 neuron tương ứng 10 lớp của CIFAR-10.

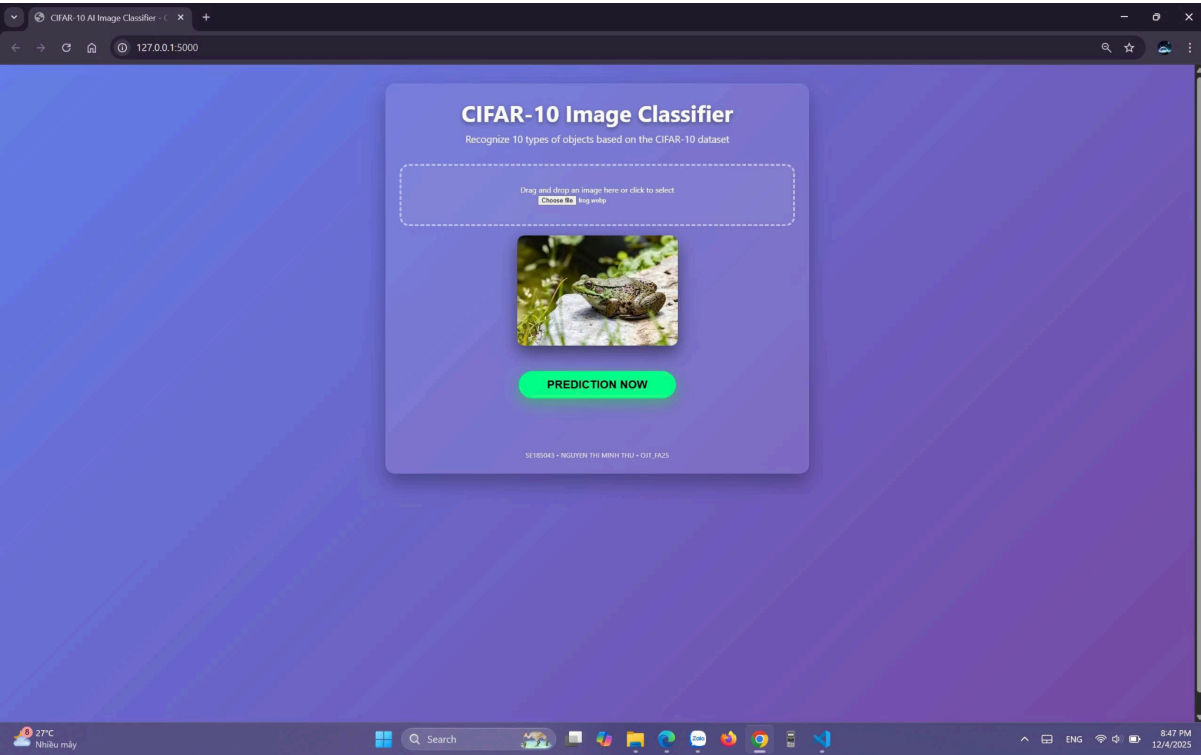
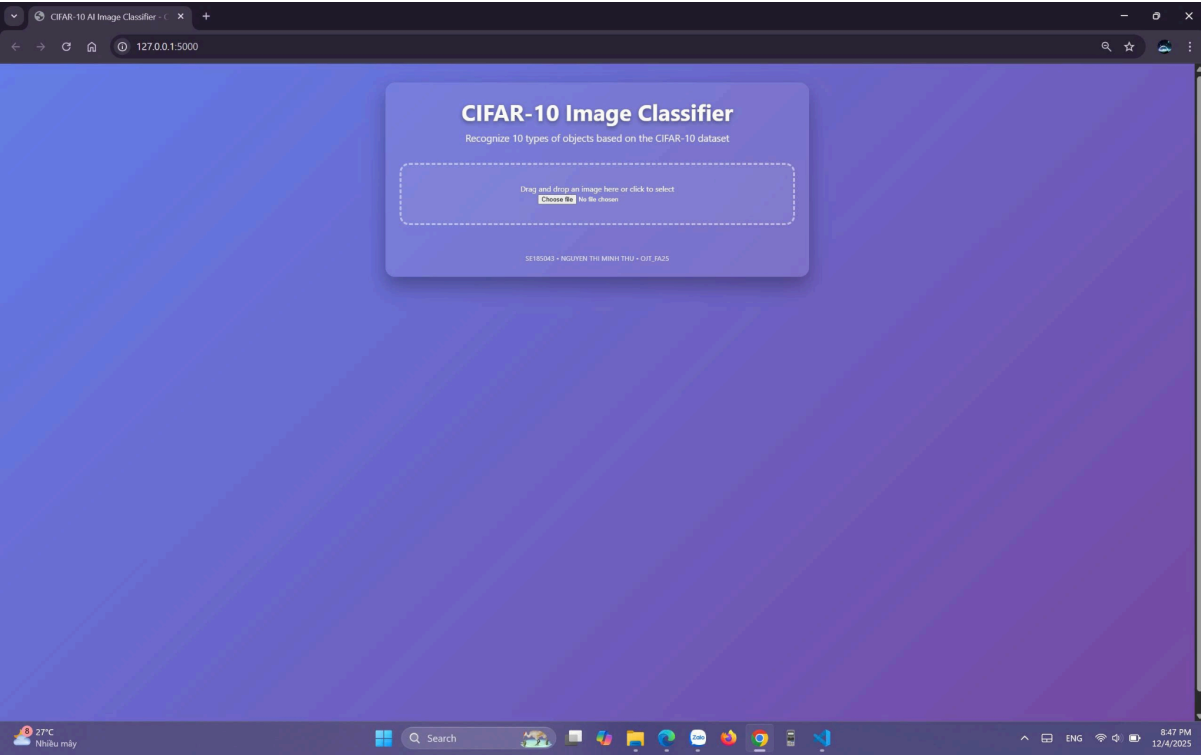
Softmax activation: chuẩn hóa đầu ra thành phân phối xác suất, cho phép mô hình dự đoán lớp có xác suất cao nhất.

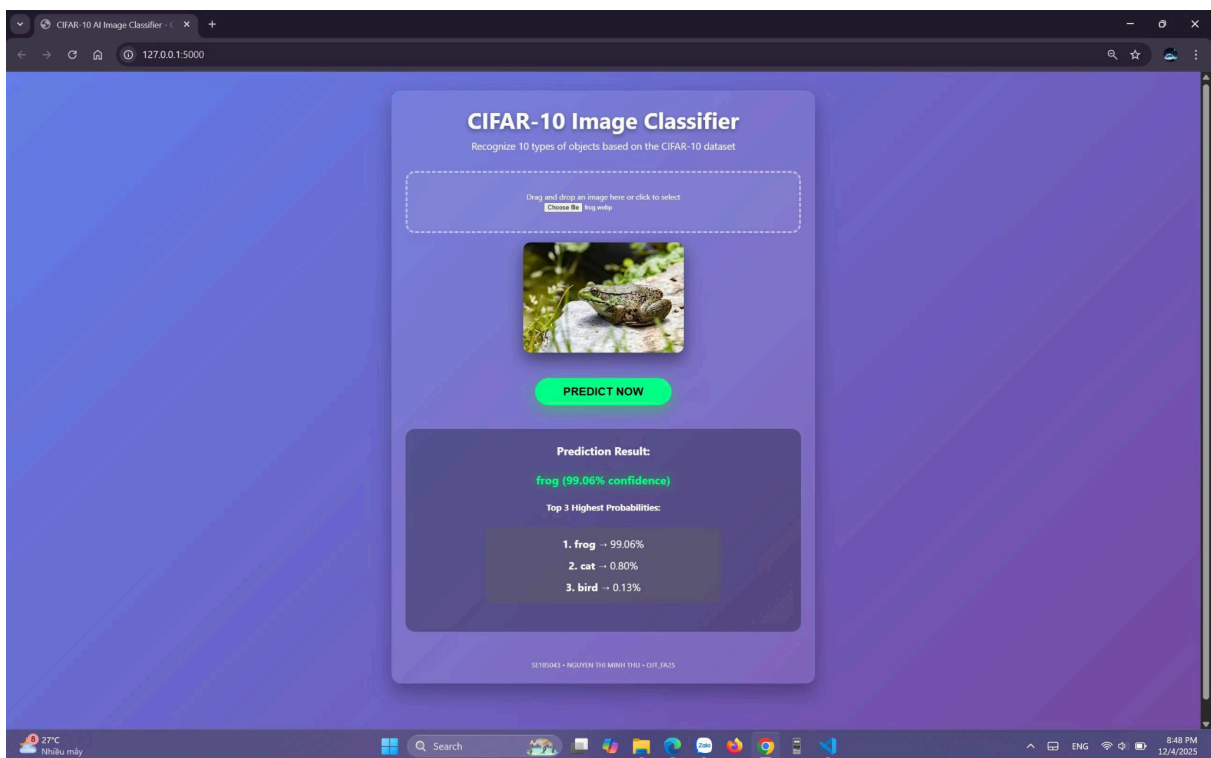
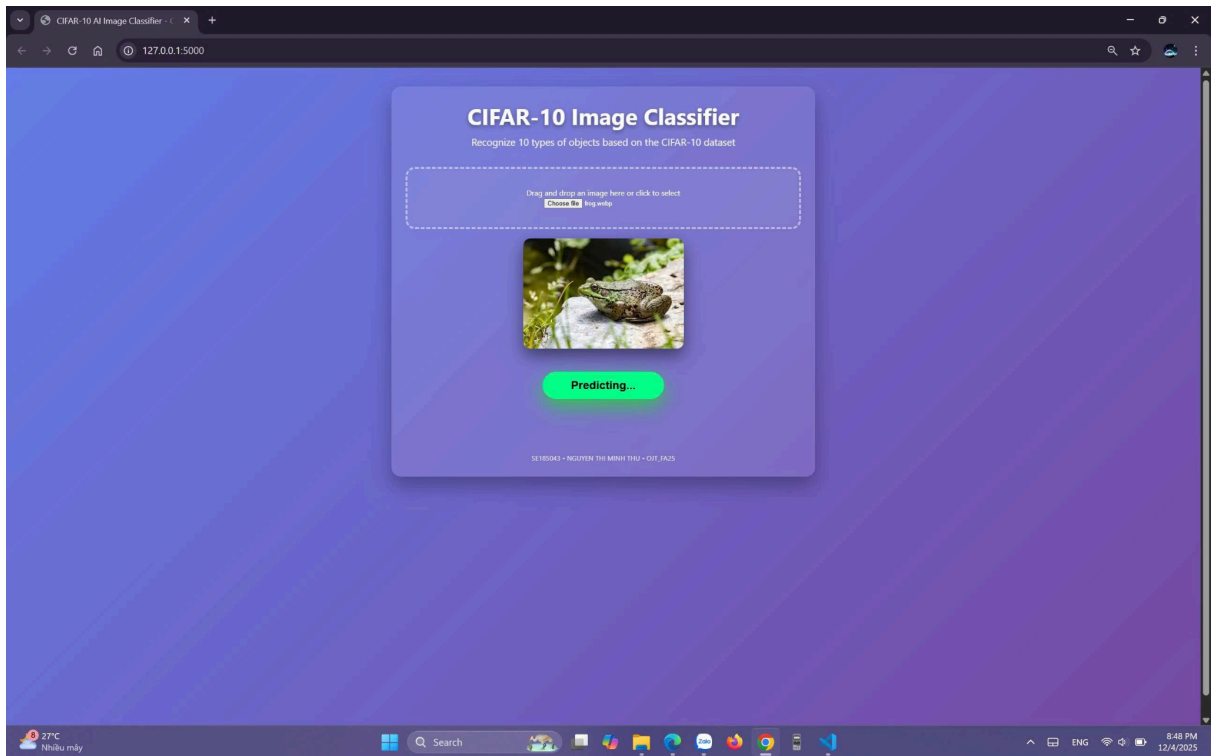
3.3. Quy trình huấn luyện

Trong quá trình huấn luyện, bộ tối ưu Adam được sử dụng với tốc độ học (learning rate) được thiết lập ở mức 5×10^{-4} . Hàm mất mát được lựa chọn là Categorical Cross-Entropy, phù hợp cho các bài toán phân loại đa lớp. Để cải thiện hiệu quả huấn luyện và tránh hiện tượng overfitting, áp dụng nhiều callback hỗ trợ: ReduceLROnPlateau với hệ số giảm 0.5 và ngưỡng kiên nhẫn 10 epoch, EarlyStopping với ngưỡng kiên nhẫn 40 epoch cùng cơ chế khôi phục trọng số tốt nhất, và ModelCheckpoint nhằm lưu lại mô hình tốt nhất cũng như bản sao ở mỗi epoch. Ngoài ra, hệ thống còn được tích hợp cơ chế tự động phát hiện và tiếp tục huấn luyện từ checkpoint mới nhất, đảm bảo quá trình huấn luyện không bị gián đoạn và có thể khôi phục dễ dàng khi cần thiết.

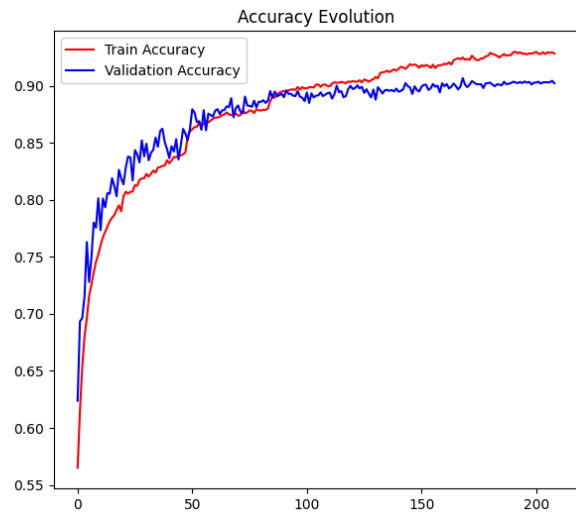
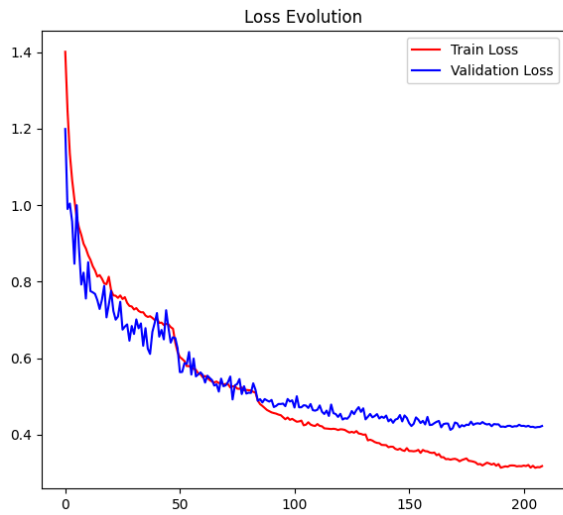
3.4. Triển khai ứng dụng web

Hệ thống được xây dựng với Backend sử dụng Flask, đảm bảo khả năng xử lý dữ liệu và triển khai mô hình dự đoán. Frontend được phát triển bằng HTML5 kết hợp CSS3 theo phong cách glassmorphism cùng với Vanilla JavaScript, mang lại giao diện hiện đại và trực quan. Ứng dụng hỗ trợ người dùng kéo-thả hoặc lựa chọn ảnh từ thiết bị, đồng thời cung cấp chức năng xem trước tức thì. Quá trình dự đoán diễn ra trong thời gian thực với độ trễ dưới 1 giây, kết quả được hiển thị dưới dạng top-3 xác suất cao nhất, giúp người dùng dễ dàng đánh giá và đối chiếu.





4. KẾT QUẢ VÀ ĐÁNH GIÁ



Chỉ số	Giá trị
Độ chính xác tập test	90.70%
Độ chính xác validation tốt nhất	92.18%
Loss tập test	0.4241
Số epoch thực hiện	209 (tự dừng sớm)

Biểu đồ học (learning curves) cho thấy không có hiện tượng overfitting đáng kể. Mô hình duy trì khoảng cách nhỏ giữa train/val accuracy trong toàn bộ quá trình huấn luyện.

Dự đoán trên ảnh thực tế

CIFAR-10 PREDICTION RESULT



Kết quả chứng minh khả năng tổng quát hóa tốt của mô hình.

5. KẾT LUẬN

Dự án đã hoàn thành tốt các mục tiêu đề ra:

- ☒ Đạt độ chính xác cao thuộc top đầu với kiến trúc tự xây dựng
- ☒ Xây dựng quy trình huấn luyện bền vững, có khả năng chịu lỗi
- ☒ Phát triển ứng dụng web hoàn chỉnh, sẵn sàng demo và triển khai thực tế
- ☒ Đáp ứng đầy đủ yêu cầu của chương trình OJT từ khâu nghiên cứu đến sản phẩm

Công trình này không chỉ là một bài tập học thuật mà là một sản phẩm hoàn chỉnh có thể áp dụng ngay trong các hệ thống giám sát giao thông, an ninh hoặc nhận diện đối tượng tự động.

6. HƯỚNG PHÁT TRIỂN

1. Áp dụng các kiến trúc hiện đại (EfficientNet-B0, ResNet-50) để đạt hiệu suất cao hơn
2. Mở rộng bộ dữ liệu với các lớp đặc thù Việt Nam (xe máy, người đi đường)
3. Triển khai real-time detection từ webcam
4. Chuyển đổi sang TensorFlow Lite để tích hợp trên thiết bị di động
5. Đóng gói Docker và triển khai trên nền tảng cloud (AWS/GCP)

TÀI LIỆU THAM KHẢO

1. [Krizhevsky, A. \(2009\). Learning Multiple Layers of Features from Tiny Images. Technical Report, University of Toronto.](#)

2. [Simonyan, K., & Zisserman, A. \(2015\). Very Deep Convolutional Networks for Large-Scale Image Recognition. ICLR 2015.](#)
3. [Chollet, F. \(2017\). Deep Learning with Python. Manning Publications.](#)